
boolean



PRISMA

ORM per Node.js

Prisma JS

Cosa sono gli ORM?

Prisma è un **ORM** (Object-Relational Mapping) open source progettato per applicazioni Node.js.

Un **ORM** è uno strato di astrazione che permette allo sviluppatore di interagire con il database senza doversi preoccupare del suo sistema sottostante.

Nella pratica, gli **ORM** sono delle **librerie**, disponibili in praticamente tutti i linguaggi di programmazione, che **trasformano le tabelle del database in oggetti e le operazioni** (come select, insert, update e delete) **in metodi**.

Prisma JS

Le principali caratteristiche

- **Compatibilità con diversi DBMS:** Prisma supporta una varietà di database relazionali, tra cui MySql, PostgreSQL, e MongoDB.
- **Semplicità nella definizione di tabelle e campi:** Prisma offre una sintassi intuitiva per definire lo schema del database, semplificando la creazione dei modelli dei dati.
- **Query Builder:** Prisma fornisce un Query Builder tramite il Prisma Client, consentendo di eseguire query in modo semplice e intuitivo senza richiedere la scrittura di stringhe SQL le quali sono spesso verbose e soggette a errori di scrittura.
- **Migrations del database:** Le migrazioni consentono di aggiornare lo schema del database in modo controllato e tracciabile, tenendo traccia delle modifiche apportate nel tempo.





PRISMA

Installazione & Configurazione

Prisma

Installazione e configurazione

Nel terminale, all'interno del nostro progetto Node.js, eseguiamo i seguenti comandi:

```
pnpm add --save-dev prisma@7 dotenv
```

Per installare la **CLI** di **Prisma**. e abilitare la lettura delle variabili file `.env` nei file di configurazione.

```
pnpm add @prisma/client@7 @prisma/adapter-mariadb
```

Per installare il **Prisma Client** e l'adapter mariadb per lavorare con **MySQL**

Prisma

Installazione e configurazione

Ora procediamo con l'**inizializzazione** di Prisma, utilizzando **MySQL** come DBMS.

Eseguiamo il seguente comando nel terminale:

```
npx prisma init --datasource-provider mysql
```

Al termine di questa operazione troveremo:

- un file di configurazione **`prisma7.config.ts` al quale cambieremo l'estensione del file in `.js`**

- una cartella **`prisma`** nella root del nostro progetto, con un file **schema.prisma** al suo interno.

- una cartella **`.agents`** con le skill per Prisma.

Inoltre, se non era già presente, verrà creato un file **.env** con una variabile d'ambiente chiamata **DATABASE_URL** che definisce l'URL di connessione al database.

Prisma

Il file di configurazione

prisma7.config è il **file di configurazione di Prisma** in cui vengono definiti i settaggi per il file di schema, le migrazioni e la sorgente dati. Questo file viene generato nella root del nostro progetto.

Il file viene creato di default con estansione **.ts** (TypeScript) e dobbiamo ricordarci di cambiare questa estensione di file in **.js** in quanto utilizziamo JavaScript nella nostra app Node.

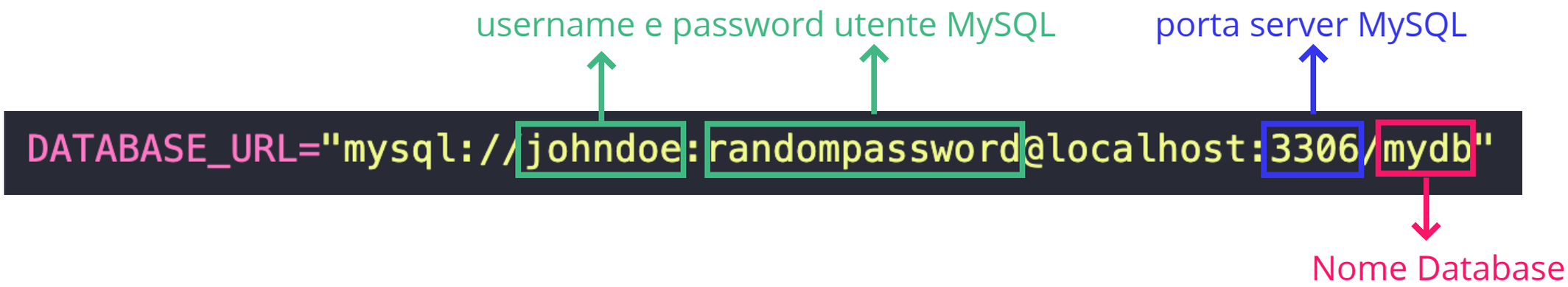
```
// In questo file di configurazione non vengono caricate
// in automatico le variabili del file .env
import 'dotenv/config';
import { defineConfig } from 'prisma/config';

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
    seed: 'node prisma/seed.js',
  },
  datasource: {
    // credenziali al nostro DB MySQL
    url: process.env.DATABASE_URL,
  },
});
```

Prisma

Configurazione url di connessione

Modifichiamo l'URL di connessione inserendo le informazioni del nostro server MySQL locale nel file **`.env`**



Nota: Non è necessario creare manualmente il database, Prisma si occuperà di questo automaticamente quando eseguiremo la prima migration. 😉

Prisma

Il file schema.prisma

`schema.prisma` è il file in cui definiremo la **struttura** del nostro database, attraverso i **Modelli.**

All'interno del file, notiamo che è già presente un contenuto di setup iniziale:

- generator client genera il client, ovvero, lo strumento che ci consentirà di interagire con il nostro database.
Qui **assicuriamoci di avere `prisma-client-js`** per utilizzare la versione JavaScript e non TypeScript.

- datasource db definisce il database utilizzato, nel nostro caso MySql.

Le credenziali del DB vengono recuperate dal file di configurazione **`prisma7.config.js`** visto nella slide precedente.

```
generator client {
  provider = "prisma-client-js"
  output   = "../generated/prisma"
}

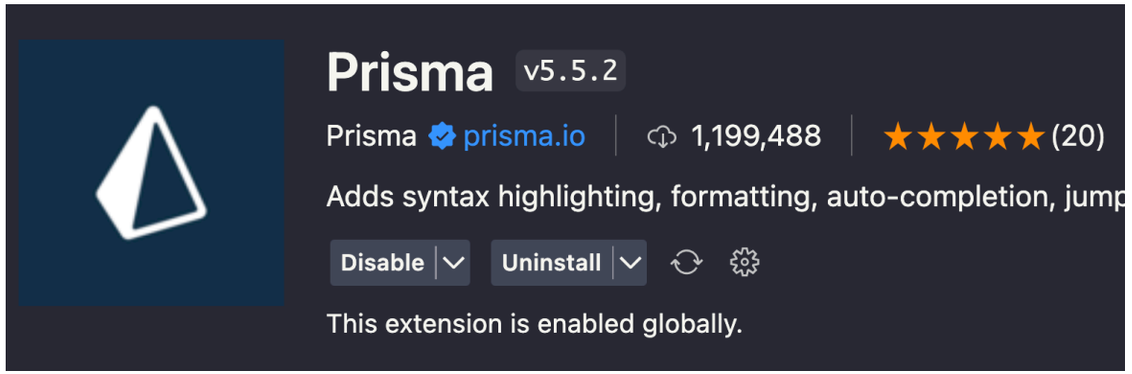
datasource db {
  provider = "mysql"
}
```

Setup VSCode / Antigravity

Aggiungiamo l'estensione Prisma

Questa estensione facilita l'uso della sintassi di Prisma.

Mostra la sintassi colorata e fornisce l'auto-completamento delle istruzioni 😎



Per abilitare l'auto-formattazione al salvataggio dei file Prisma, è sufficiente aggiungere questa configurazione al file **settings.json** di VSCode.

```
"[prisma]": {
  "editor.defaultFormatter": "Prisma.prisma",
  "editor.formatOnSave": true
},
```



LIVE CODING

Installiamo e Configuriamo Prisma

Modellazione dei Dati

Come creare le tabelle nel nostro database

Modellazione dei Dati

Creiamo le tabelle del nostro DB

Come menzionato in precedenza, Prisma offre un'estensione e una sintassi specifica per definire la struttura del nostro database.

In linea con gli approcci tradizionali degli ORM, Prisma ci consente di definire **modelli** che rappresentano le **entità della nostra applicazione**, ovvero le **tabelle** del nostro database insieme ai relativi campi (**colonne** della tabella) e alle possibili relazioni tra di esse.

Definizione di un modello

Per illustrare questo concetto, definiamo il nostro primo modello `Product`, che conterrà inizialmente i seguenti campi:

- **id** (Numero intero)
- **name** (Stringa)
- **description** (Stringa)
- **price** (Numero decimale)
- **quantity** (Numero Intero)
- **isActive** (Booleano)
- **createdAt** (DateTime)
- **updatedAt** (DateTime)

Prisma

Modello Product

Di norma, nel contesto di Prisma, definiamo **i nomi dei modelli al singolare e** utilizziamo la notazione **PascalCase**. Inoltre, è obbligatorio includere un campo che funga da **chiave primaria** all'interno del modello.

```
// schema.prisma

generator client {
  provider = "prisma-client-js"
  output   = "../generated/prisma"
}

datasource db {
  provider = "mysql"
}

model Product {
  id    Int    @id @default(autoincrement())
}
```

Il campo **id** sarà la chiave primaria (**PK**) della tabella 'Product' ed è di tipo **intero** con **auto-incremento**.

A breve esamineremo questa sintassi nel dettaglio 😉

https://www.prisma.io/docs/orm/v7/prisma-schema/data-model/models

Prisma

Definizione dei campi

I **campi** definiti all'interno di un modello rappresentano le **colonne nella tabella** del database.

Ogni campo è caratterizzato da:

- **nome**: in formato camelCase
- **tipo di dato**: String, Int, Float, Boolean ecc... *lista completa*
- **modificatore (opzionale)**:
  - [] per indicare che il campo contiene una lista, ad esempio per le relazioni tra tabelle.
  - ? per indicare che il campo è opzionale.
- **attributi (opzionali)**:
  - @id per definire il campo come chiave primaria.
  - @default per assegnare un valore predefinito al campo.
  - @unique per specificare che il campo deve essere unico, quindi non possono esistere due record con lo stesso valore.

Prisma

Definizione dei campi per il modello Product

Ora procediamo all'aggiunta dei campi al modello che abbiamo definito in precedenza.

Inoltre, includiamo due campi di tipo DateTime denominati '**createdAt**' e '**updatedAt**' per memorizzare la data di creazione di una pizza e l'ultima modifica apportata alla stessa.

```prisma
// schema.prisma

model Product {
  id          Int      @id @default(autoincrement()) /
  name        String
  description String?  @db.Text
  price       Decimal  @db.Decimal(8, 2)
  quantity    Int      @default(0)
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```



LIVE CODING

Creiamo il modello Product



MIGRATIONS

Come modificare il database

Prisma

Le migrations

Ora potreste chiedervi: come rendere effettive queste modifiche e creare il database con la tabella 'Product'? 🤔

Prisma dispone di un **sistema di migrazioni che tiene traccia delle modifiche apportate al database**.

Ogni volta che desideriamo apportare modifiche al nostro database dovremmo:

- modificare il file **schema.prisma.**
- generare ed eseguire una migrazione.

Una migrazione è essenzialmente un file SQL che contiene le query per apportare le modifiche al nostro database.

Prisma

La prima migration 🥳

Per generare ed eseguire una migration usiamo il seguente comando:

```
npx prisma migrate dev --name add_product
```

L'opzione --name non è obbligatoria e permette di assegnare un nome alla migration

⚠️ Attenzione

Dopo la modifica al database con una migration dobbiamo ricordarci di aggiornare il Prisma Client, il quale ci permetterà di eseguire le queries al nostro db, ad esmepio per le CRUD. Assicuriamoci quindi che il **Prisma Client sia sempre aggiornato con l'ultima versione dello schema DB**.

```
npx prisma generate
```



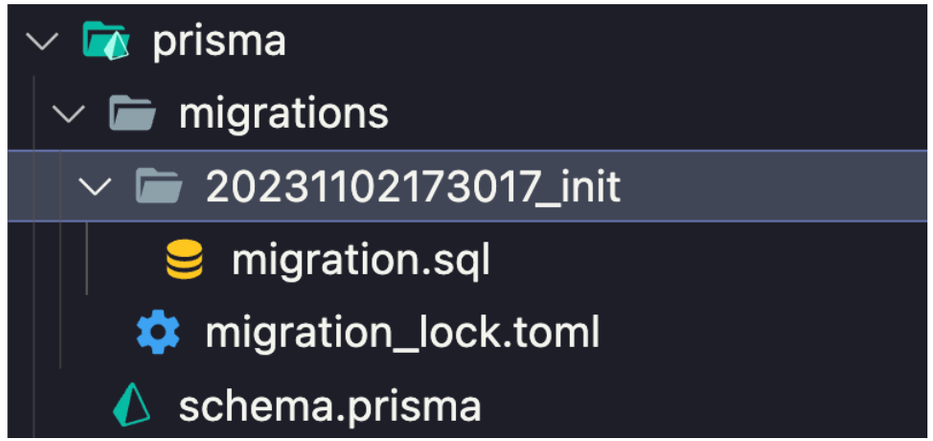
Se vogliamo rimuovere tutti i dati dal DB e rieseguire tutte le migration create esiste anche il comando `migrate reset`

```
npx prisma migrate reset
```

Prisma

Che cosa genera una migration

Quando generiamo una migrazione, viene creata una cartella all'interno della directory '**prisma/migrations**'.



All'interno di questa cartella troveremo un file SQL che contiene le query per apportare modifiche allo schema del database.

Per registrare e tenere traccia delle migrazioni eseguite, Prisma crea una tabella chiamata **_prisma_migrations** all'interno del database.

id	checksum	finished_at	migration_name	logs	rolled_back_at	started_at	applied_steps_count
a330d62e-2f18-4a40-951e-37baf4a1981c	ec6c6e2102786268b36dc09	2023-11-02 17:31:40.919	20231102173017_init	*NULL*	*NULL*	2023-11-02 17:31:40.908	1



LIVE CODING

Eseguiamo la prima migration



RELAZIONI

Aggiungiamo le relazioni fra i nostri modelli

Relazioni

Intro

Prisma ci offre la possibilità di definire facilmente le classiche relazioni come: **one-to-one**, **one-to-many** e **many-to-many**.

Cominciamo aggiungendo il modello **Category**, che sarà collegato al modello **Product** attraverso una relazione **one-to-many**.

- ogni **Prodotto** può o deve essere associato a una **Categoria**
- ogni **Categoria** può essere associata a più **Prodotti**.

https://www.prisma.io/docs/orm/v7/prisma-schema/data-model/relations

Relazioni

One to Many

Modifichiamo lo schema.prisma aggiungendo il modello **Category**.

All'interno di questo modello, aggiungiamo il campo **products**, il cui tipo di dato sarà il nome del modello **Product**, con il modificatore **[]** per indicare che una categoria può essere associata a molti prodotti.

```prisma
model Category {
  id         Int      @id @default(autoincrement())
  name       String
  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt
  products   Product[]
}
```

Relazioni

One to Many: foreign key

Modifichiamo anche il modello **Product** includendo la foreign key **categoryId** e il vincolo di relazione col modello **Category**.

```prisma
model Product {
  id          Int      @id @default(autoincrement()) // PK
  name        String
  description String?  @db.Text
  price       Decimal  @db.Decimal(8, 2)
  quantity    Int      @default(0)
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  category    Category @relation(fields: [categoryId], references: [id])
  categoryId  Int // FK
}
```

Relazioni

One to Many: Relazione opzionale

Per rendere la relazione opzionale, è sufficiente aggiungere il modificatore **?** accanto al tipo di dato sia del vincolo che della foreign key.

```
model Product {
  id          Int      @id @default(autoincrement()) // PK
  name        String
  description String?  @db.Text
  price       Decimal  @db.Decimal(8, 2)
  quantity    Int      @default(0)
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  category    Category? @relation(fields: [categoryId], references: [id])
  categoryId  Int? // FK
}
```

Relazioni

One to Many

Semplice, vero? 🤓

Ma assicuratevi di eseguire la migration per applicare le modifiche al database!!

```
1 npx prisma migrate dev --name add_category
```

E di aggiornare il Prisma Client:

```
1 npx prisma generate
```



LIVE CODING

One to Many

Relazioni

Many to Many

Procediamo con la modifica dello schema.prisma, questa volta aggiungendo il modello **Tag**.

Come fatto in precedenza, aggiungiamo il campo **products**, il cui tipo di dato sarà il nome del modello **Product** con il modificatore **[]**

```
model Tag {
  id         Int      @id @default(autoincrement())
  name       String
  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt
  products   Product[]
}
```

Relazioni

Many to Many

Aggiungiamo ora il campo **tags** al modello **Product**, continuando ad utilizzare il modificatore **[]**

```
model Product {
  id          Int      @id @default(autoincrement()) // PK
  name        String
  description String?  @db.Text
  price       Decimal  @db.Decimal(8, 2)
  quantity    Int      @default(0)
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  category    Category @relation(fields: [categoryId], references: [id])
  categoryId  Int
  tags Tag[]
}
```

Relazioni

Many to Many: tabella Pivot

E la tabella ponte???

Nessun problema, Prisma si occuperà di tutto quando verrà eseguita la migrazione!

😎



DB Seeding

Aggiungere dati di esempio al DB

DB Seed

Aggiungere dati di esempio nel DB

Il **seed** in Prisma è uno script JavaScript, da eseguire via terminale, che **popola il database con dati iniziali** (o di test), ad esempio prodotti di prova, categorie, ecc...

Come primo step dobbiamo dichiarare il comando `seed` nel file **prisma7.config.js**. Questo comando permettera di eseguire lo script seed da terminale.

```
1  import 'dotenv/config';
2  import { defineConfig } from 'prisma/config';
3
4  export default defineConfig({
5    schema: 'prisma/schema.prisma',
6    migrations: {
7      path: 'prisma/migrations',
8      seed: 'node prisma/seed.js',
9    },
10   datasource: {
11     url: process.env.DATABASE_URL,
12   },
13 });
```

DB Seed

Aggiungere dati di esempio nel DB

Ora creaiamo nella cartella **`prisma`** il file **`seed.js`**

In questo file:

1 Impostiamo il prisma client con relativo adapter per MySQL

2 Predisponiamo la main function che conterrà la logica del nostro seed

```
1  import 'dotenv/config';
2  import { PrismaMariaDb } from '@prisma/adapter-mariadb';
3  import { PrismaClient } from '../generated/prisma/index.js';
4
5  // Setup client
6  const adapter = new PrismaMariaDb(process.env.DATABASE_URL);
7  const prisma = new PrismaClient({ adapter });
8
9  // Seed logic function
10 async function main() {
11   console.log('Seeding started');
12 }
13
14 main()
15   .catch(e => {
16     console.log(e);
17     process.exit(1); // termina il processo nel terminale
18   })
19   .finally(() => prisma.$disconnect()); // chiude connessione al DB
```

Per eseguire il file **`seed.js`** eseguiamo il seguente comando nel terminale:

```
1  npx prisma db seed
```

Prisma JS

Come utilizzare il Prisma Client

Per specificare su quale modello desideriamo effettuare le operazioni, utilizziamo il **nome del modello** (in formato camelCase) come proprietà dell'oggetto istanziato dalla classe PrismaClient.

Nel nostro caso utilizzeremo ad esempio **`prisma.product`**, seguito dal nome del metodo che corrisponde al tipo di operazione che desideriamo eseguire.

Tutte le operazioni che eseguiamo sul database tramite il Prisma Client sono **asincrone**.

DB Seed

Aggiungere dati di esempio nel DB

All'interno della funzione **`main()`** possiamo ora definire la logica per inserire dati nel DB grazie all'utilizzo del Prisma Client **`prisma`**

Notiamo l'inserimento della foreign key **`categoryId`** durante la creazione dei prodotti utilizzando una categoria randomica tra quelle create nell'array **`categories`**

```
1  // All'interno della funzione main()
2
3  // Rimozione eventuali dati esistenti
4  await prisma.product.deleteMany();
5  await prisma.category.deleteMany();
6
7  // Creazione categorie
8  const categoryNames = ['Lorem', 'Ipsum', 'Amet'];
9
10 await prisma.category.createMany({
11     data: categoryNames.map(cat => ({
12       name: cat,
13     })),
14 });
15
16 // Lettura delle catgorie appena create
17 const categories = await prisma.category.findMany();
18 console.log(categories);
19
20 // Creazione prodotti
21 const products = [];
22 for (let index = 0; index < 10; index++) {
23   products.push({
24     name: `Product ${index + 1}`,
25     price: 10.55,
26     categoryId: categories[randIndex(categories.length)].id,
27   });
28 }
29
30 await prisma.product.createMany({
31   data: products,
32 });
```



ESERCITAZIONE